SENDA-TR-2026-01 · TECHNICAL REPORT

Proof-Carrying Inference for Offline Emergency Software

A checkable evidence envelope for numerical claims

Abraham Haddioui · 2026-11-07 · version 1.0

Technical report · not peer reviewed · MIT

Abstract

Emergency software often presents numbers—delivery probabilities, credible intervals, delay bounds, mass-conservation errors—without a portable way to check how those numbers were produced. This report specifies **PCI v0.1 (Proof-Carrying Inference)**, a small JSON envelope that binds a displayed number to its inputs, method, tolerances, provenance class, limitations, and a SHA-256 seal. A verifier checks four things offline: the structure, the digests, the display rule, and the provenance evidence; an optional ECDSA P-256 signature adds signer binding. We state what the envelope proves and what it does not. The central negative result is simple: a valid seal does not prove that the producer executed the declared method. PCI is an evidence format, not a proof of computational integrity. We then give a positive result for cross-runtime display determinism: if a claim uses an upper-bound display and every observed raw value lies below the bound, the rendered text is identical across runtimes even when the raw floating-point values differ. We illustrate the format with a SEIR integration whose raw mass error differs between Node.js and Chrome while the published display remains `< 1e-9`. The reference implementation is a dependency-free MIT-licensed package with 17 tests, a command-line verifier, and an optional signature profile.

Keywords: offline systems, provenance, verification, numerical software, emergency computing, reproducibility.

1. Introduction

A field application may show that a message has a 97.3% chance of delivery within twelve hours, or that an adaptive integrator conserves mass to better than 10^{-9}. An operator can act on the number, but a reviewer cannot reconstruct it from a screenshot. The inputs, the model version, the tolerance, the rounding rule, and the epistemic status of the number are usually scattered across code, build logs, and documentation—if they exist at all.

This report asks a narrower question than “is the model correct?”:

> *Can a third party check, without network access, that a displayed number is the number the producer committed to, and that the producer declared the conditions under which it should be read?*

PCI answers that question with an envelope, a canonical byte encoding, three SHA-256 digests, a display contract, and a provenance class. The design is deliberately small enough to reimplement. It is not a consensus protocol, a zero-knowledge proof, or a replacement for field trials.

The contributions are:

1. a precise claim model and canonicalization rules for numerical claims;
2. a four-level verification procedure with explicit failure semantics;
3. a display contract that separates raw floating-point values from the text a human sees;
4. a cross-runtime determinism result for upper-bound displays;
5. an open reference implementation and a documented negative result about what sealing does not prove.

Section 2 states requirements. Section 3 defines the envelope and its integrity properties. Section 4 analyzes display determinism. Section 5 gives the threat model. Section 6 describes the implementation and case study. Section 7 relates the work to existing attestation and provenance formats. Section 8 lists limitations and non-claims. Section 9 concludes.

---

2. Requirements

We derived the following requirements from offline emergency deployments:

- **R1 — Offline verification.** Verification must use only the claim and an optional public key. No registry, timestamp authority, or network call.
- **R2 — Explicit inputs.** Every input that changes the result must be present or committed by digest.
- **R3 — Explicit method and tolerance.** The claim must name the method and declare whether it is exact; numerical methods must expose tolerances.
- **R4 — Display contract.** The human-visible text must be recomputable from the raw value and a declared rule.
- **R5 — Epistemic status.** Exercise priors, simulations, field observations, validated observations, and derived claims must be distinguishable.
- **R6 — Tamper evidence.** A change to the payload must invalidate the seal under a standard collision-resistance assumption.
- **R7 — Small surface.** A competent developer should be able to implement a verifier from the specification in a day.

R1 rules out designs that depend on an online transparency log for the base check. R4 is the requirement most often missing from provenance systems: a hash of a float is not a statement about what a human was shown.

---

3. The PCI envelope

3.1 Claim tuple

A claim is a tuple

$$C = (m, x, M, r, \delta, p, \ell, q),$$

where m identifies the model and version, x is the input object, M is the method and its tolerances, r is the raw result, δ is the display rule, p is the provenance record, ℓ is a non-empty list of limitations, and q is the reproduction record. The wire format adds a version string, a claim identifier, an integrity object, and an optional signature.

3.2 Canonical encoding

Let $\kappa(C)$ be the canonical JSON encoding: NFC-normalized strings, sorted object keys, no insignificant whitespace, finite numbers serialized by the ECMAScript algorithm, and rejection of non-JSON values. This is close to RFC 8785 [1], restricted to the JSON subset PCI needs. Canonicalization makes the digest independent of property order and Unicode composition.

The payload is C with `integrity` and `signature` removed. The sealer computes

$$\sigma(C) = \left(H(\kappa(C_{\text{payload}})),\ H(\kappa(x)),\ H(\kappa(r))\right),$$

where H is SHA-256 [2]. The three digests bind the whole claim, the inputs, and the result separately. The separate digests make failures diagnosable: a changed input invalidates both the payload and input digests, while a changed display invalidates the payload and result digests.

3.3 Verification predicate

A verifier returns a level in $\{0, 1, 2, 3, 4\}$ and a Boolean. The levels are:

- **L0:** structure and types;
- **L1:** all three digests;

- **L2:** display text equals the text produced by $\delta(r)$;
- **L3:** provenance requirements;
- **L4:** ECDSA P-256 signature, when a public key is supplied.

A verifier must not report L2 if L1 failed. A signature present without a public key produces a warning, not L4.

3.4 What the seal proves

Proposition 1 (tamper evidence). Assume SHA-256 is collision-resistant. If two claims C and C' differ in any payload field, then with overwhelming probability $\sigma(C) \neq \sigma(C')$.

This is the standard reduction: a collision in the payload digest yields a SHA-256 collision. The proposition is conditional on the hash assumption and on the verifier using the same canonicalization.

Proposition 2 (display soundness). If verification reaches L2, then the claim's text is exactly the text generated by the declared rule applied to the raw value.

This is by construction: L2 requires byte-for-byte equality with the expected string. It says nothing about whether the rule is a good way to present the number.

Proposition 3 (signature binding). If the ECDSA verification succeeds, the signer possessed the private key corresponding to the supplied public key and signed the payload digest.

This is the standard unforgeability property of ECDSA [3]. It does not establish that the signer is trustworthy, that the key was not stolen, or that the payload is true.

3.5 What the seal does not prove

Proposition 4 (no execution proof). A valid PCI claim does not imply that the producer executed the declared method.

Proof. A producer can compute any value r, construct a well-formed claim with arbitrary inputs and a declared method, and seal it. All verifier checks are local functions of the claim and the key. Nothing in the envelope binds the claim to an execution trace. Therefore a valid claim is consistent with both faithful and fabricated execution. ■

This is not a defect to be patched with more hashing. Closing the gap requires trusted execution, reproducible builds with independent re-execution, or a cryptographic proof system with a cost and trust model that most offline emergency deployments cannot carry. PCI chooses the cheaper, narrower claim.

4. Display determinism across runtimes

4.1 The problem

Floating-point results can differ across JavaScript engines because transcendental functions such as `Math.pow` are implementation-approximated, and because engines may optimize arithmetic differently. An adaptive integrator turns a tiny difference in an error estimate into a different step-size trajectory. Hashing the raw float would make the claim runtime-specific and fragile. Hashing only the displayed text would hide the raw uncertainty.

PCI separates the two. The raw value is committed, and the display rule states what the human sees. For quantities that are known only up to a bound, the correct display is a bound, not a fixed number.

Definition 1 (display determinism). Let R be a set of runtimes and let v_ρ be the raw value produced by runtime $\rho \in R$. A display rule δ is deterministic over R if $\delta(v_\rho)$ is the same string for all $\rho \in R$.

Lemma 1 (upper-bound determinism). Let δ be the rule `upper-bound` with bound $B > 0$, rendering `< B` when $v < B$ and `≤ B` when $v \le B$. If every v_ρ satisfies $0 \le v_\rho < B$, then δ is deterministic over R and renders `< B`.

Proof. The rule's output depends only on the predicate $v < B$ and on the constant B, not on the numerical value of v. Every runtime satisfies the predicate, so every runtime produces the same string. ∎

The lemma is deliberately modest. It does not say the raw values are equal; it says the published text is a stable contract. The raw values remain in the claim for audit.

For fixed-point displays, a sufficient condition is a shared rounding interval.

Lemma 2 (fixed-point margin). Let δ be `fixed:N`. If there exists an integer k such that both v and v' lie strictly inside $\left((k-\tfrac{1}{2})10^{-N}, (k+\tfrac{1}{2})10^{-N}\right)$, then $\mathrm{toFixed}(v, N) = \mathrm{toFixed}(v', N)$.

This follows because `toFixed(N)` rounds to the nearest multiple of 10^{-N}; the open interval excludes the tie boundaries. In practice a producer should require a margin much larger than the observed cross-runtime spread, or use an upper bound.

4.2 Observed case

The SENDA precision lab integrates a four-compartment SEIR model with the Dormand–Prince 5(4) pair [4]. The exact ODE preserves $S + E + I + R = N$; the numerical trajectory has a mass defect

$$e(t) = \left| S(t) + E(t) + I(t) + R(t) - N \right| .$$

The lab reports $\max_t e(t)$ over daily samples. With population $N = 1000$, eight contacts per day, transmission probability $\beta = 0.15$, three latent days, five infectious days, and ninety simulated days, the observed values were:

RUNTIME	VERSION	RAW MAX MASS ERROR	DISPLAY	ACCEPTED STEPS
Node.js	22.22.2	$1.8189894035458565 \times 10^{-12}$	`< 1e-9`	233
Google Chrome (headless)	156.0.0.0	$1.0231815394945443 \times 10^{-12}$	`< 1e-9`	233

The raw values differ by about 7.96×10^{-13}, yet both satisfy $e < 10^{-9}$. By Lemma 1 the display is deterministic. The accepted step count also agreed in this run, but that is an observation, not a guarantee. A matrix of four parameter settings is reported in SENDA-TR-2026-03.

This example motivates the design: publish the bound that matters to the operator, commit the raw value for audit, and do not pretend that a float hash is a cross-runtime display contract.

---

5. Threat model

THREAT	PCI RESPONSE	RESIDUAL RISK
Payload modified at rest	Three SHA-256 digests	Hash collision; canonicalization bug
Inputs swapped between claims	Input digest and claim ID	Producer fabricates inputs
Display edited in a screenshot	Display rule and text check	Screenshot can omit the claim
Prior presented as field data	Provenance class and source digest	Source itself may be bad
Claim replayed in another context	Claim ID and model binding	No context field in v0.1
Signature forged	ECDSA P-256	Key theft; weak key management
Compromised runtime	None	Producer can lie before sealing
Verifier bug	Test vectors and independent implementations	Implementation diversity is immature

The threat model is intentionally local. PCI is useful when the adversary can modify files or messages but does not control the verifier and cannot break the hash or signature. It is not useful against a compromised device that is authorized to produce claims.

---

6. Implementation and case study

The reference implementation is `@senda/pci` 0.1.0, MIT-licensed, with no runtime dependencies. It uses Web Crypto for SHA-256 and ECDSA. The public API includes `canonicalize`, `sealClaim`, `verifyClaim`, `generateKeyPair`, `signClaim`, `verifyClaimSignature`, and `claimSetDigest`. The command-line verifier supports `keygen`, `seal`, `verify`, `sign`, `digest`, and `set`.

The test suite covers canonicalization, Unicode normalization, non-finite rejection, idempotent sealing, input and result tampering, display mismatch, scaled percentages, strict and inclusive bounds, integer rules, provenance requirements, signature success and failure, claim-set ordering, and a SHA-256 test vector. At the time of writing, all 17 tests pass under Node.js 22.22.2.

A reproduced example claim has payload digest

```
e205f5fbe4b9783bc0496b59601b6285ddd948407532f170898965320dde3af1
```

and verifies at L4 with a freshly generated P-256 key. Mutating the result to `1` yields two integrity mismatches and exits the verifier with code 1. The digest is deterministic across runs because the key is generated outside the payload and because sealing excludes `integrity` and `signature`.

The package does **not** claim production readiness. It is a working draft with a small API, no key-distribution story, and no independent implementation yet. Those are deliberate next-cycle deliverables, not hidden features.

---

7. Related work

PCI sits between several established lines.

Provenance. W3C PROV models entities, activities, and agents [5]. It is expressive but does not define a numeric display contract or a compact offline verifier. PCI can be embedded as a PROV entity.

Supply-chain attestations. in-toto and SLSA bind artifacts to steps and builders [6, 7]. They answer “which build produced this artifact?” PCI answers “what does this number mean, and what evidence travels with it?” The two are complementary: a PCI claim can be an in-toto subject.

Content provenance. C2PA binds assertions to media [8]. PCI targets scalar numerical claims and their display, not pixels or signatures over media.

Research objects. RO-Crate packages data, code, and metadata [9]. PCI claims can be members of an RO-Crate; the manifest in this report is a lightweight research object.

Model and dataset documentation. Model cards and datasheets describe intended use and limitations [10]. PCI makes a specific numerical claim checkable; it does not replace model documentation.

Proof-carrying code and verifiable computation. Necula's proof-carrying code attaches a machine-checkable proof to code [11]. PCI uses "proof" in the engineering sense of an evidence bundle, not a formal derivation. Cryptographic proof systems could strengthen PCI, but they change the cost and trust model.

Numerical reproducibility. Work on reproducible floating-point and round-trip formatting is directly relevant. PCI's contribution is not a new rounding algorithm; it is the envelope and the verification contract around the display.

---

8. Limitations and non-claims

- The reference implementation is not independently audited.
- ECDSA key distribution, revocation, and trust anchors are outside v0.1.
- The envelope has no context field binding a claim to a deployment, operator, or time window. A signed claim can be replayed.
- Inputs may contain personal data; PCI does not define redaction.
- The provenance class is a declaration, not a verified fact.
- The upper-bound lemma assumes the bound is correct. A wrong bound produces a deterministic but false display.
- The package does not prove execution, model validity, or operational fitness.
- The papers are technical reports, not peer-reviewed publications.

---

9. Conclusion

PCI makes a small, useful promise: a numerical claim can carry enough evidence to be checked offline, and its display can be made stable even when raw floats are not. The format is intentionally weaker than a proof of computation and stronger than an unsigned screenshot. The next steps are independent implementations, adversarial testing, and field provenance—not more claims that the seal can do something it cannot.

Availability

- Reference implementation: `packages/senda-pci` in the SENDA repository.
- Specification: `packages/senda-pci/SPEC.md`.
- Papers and downloads: `/research` on the SENDA site.
- License: MIT.
- Status: technical report, not peer reviewed.

References

[1] A. Rundgren, B. Jordan, and S. Erdtman. "JSON Canonicalization Scheme (JCS)." RFC 8785, IETF, 2020.

[2] National Institute of Standards and Technology. "Secure Hash Standard (SHS)." FIPS 180-4, 2015.

[3] National Institute of Standards and Technology. "Digital Signature Standard (DSS)." FIPS 186-5, 2023.

[4] J. R. Dormand and P. J. Prince. "A family of embedded Runge–Kutta formulae." Journal of Computational and Applied Mathematics, 6(1):19–26, 1980.

[5] W3C. "PROV-Overview: An Overview of the PROV Family of Documents." W3C Working Group Note, 2013.

[6] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos. "in-toto: Providing farm-to-table guarantees for bits and bytes." USENIX Security Symposium, 2019.

[7] Open Source Security Foundation. "SLSA: Supply-chain Levels for Software Artifacts." Specification, 2023–2026.

[8] Coalition for Content Provenance and Authenticity. "C2PA Technical Specification." 2024–2026.

[9] S. Soiland-Reyes et al. "Packaging research artefacts with RO-Crate." Data Science, 5(2), 2022.

[10] M. Mitchell et al. "Model Cards for Model Reporting." Proceedings of FAT* 2019, pp. 220–229.

[11] G. C. Necula. "Proof-Carrying Code." Proceedings of POPL 1997, pp. 106–119.

Appendix A — Reproduction

```
cd packages/senda-pci
node --test "test/*.test.mjs"
node examples/verify-claim.mjs
node bin/pci.mjs verify claim.json --public-key public.jwk.json
```

The SEIR case study is reproduced by:

```
node scripts/research/cross-runtime.mjs
```

The script writes a JSON matrix with raw values, displays, engine versions, and a pass/fail summary. It does not contact the network.